

P2505R1 Monadic Functions for

`std::expected`

Authors: Jeff Garland

Audience: LEWG, LWG

Project: ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Contact: jeff@crystalclearsoftware.com

Introduction

With the final plenary vote of [P0798 Monadic Functions for `std::optional`](#) complete, we now have a design inconsistency with `std::expected`. [P0323 `std::expected`](#) has now also been voted into the working draft for C++23. This proposal corrects the inconsistency by adding 4 functions to `std::expected` and is targeted at C++23. The author believes this should be treated as a consistency/bug fix still in scope for C++23.

The intent from P0798 clearly states that `std::expected` should gain the same functions:

There is a proposal for `std::expected` which would benefit from many of these same ideas. If the idea to add monadic interfaces to standard library classes on a case-by-case basis is chosen rather than a unified non-member function interface, then compatibility between this proposal and the `std::expected` one should be maintained

Motivation and Scope

The following 3 functions are added to `std::optional`, but are currently not part of `std::expected`.

- `and_then` compose a chain of functions returning an expected
- `or_else` returns if it has a value, otherwise it calls a function with the error type
- `transform` apply a function to change the value (and possibly the type)

After feedback on the R0, this draft also proposes the addition of a fourth function for `expected` to facilitate additional cases:

- `transform_or` apply a function to change the value, otherwise call a function with error type

For example, given the following:

```

using time_expected = expected<boost::posix_time::ptime, std::string>;

time_expected from_iso_str( std::string time )
{
    try {
        ptime t = boost::posix_time::from_iso_string( time );
        return t;
    }
    catch( std::exception& e )
    {
        return unexpected( e.what() + " str was: "s + time);
    }
}

// for use with transform
ptime next_day( boost::posix_time::ptime t )
{
    return t + boost::gregorian::days(1);
}

// for use with or_else
void print_error( std::string error )
{
    cout << error << endl;
}

//valid iso string
const std::string ts ( "20210726T000000" );

```

Before the change we'd write this:

Before

```
time_expected d = from_iso_str( ts );
if (d)
{
    ptime t = next_day( *d );
}
else
{
    print_error( d.error() );
}
```

And after, this:

After

```
// chain a series of functions until there's an error
auto d = from_iso_str ( ts )
        .or_else( print_error )
        .transform( next_day );
```

For more examples see: <https://godbolt.org/z/va5fzx11f>

Design Considerations

`expected<void, E>`

Compared with `optional`, `expected` provides the capability for `void` as a function return type. This complicates the design compared to `optional`. As currently specified the `void` specialization of `expected`, `expected<void, T>` removes the `value_or` function since `void` cannot be initialized to anything else.

While the primary implementation for this paper excludes `transform` for `expected<void, E>` and the original version removed this overload, feedback from reviewers suggested the overload is valuable. Thus the new version allows for this overload.

`transform_or` function

The R0 version of this paper did not include this function to match `optional`, but several reviewers noted the important utility of this case. Note that in the reference implementation of this paper, the function is called `map_error`.

changing of error return types

One complication of the monadic functions for `expected` as compared to `optional` is the ability to change the return error return type on subsequent function calls. `Optional`, of course, can only return `std::nullopt_t`.

Thus we would like to support use cases like the following:

```
std::expected<SomeUserType, InternalError> initialize();

std::expected<SomeUserType, UserError>
makeUserError(InternalError err)
{
    return std::unexpected( UserError(err) );
};

std::expected<SomeUserType, UserError> result = initialize().or_els
```

Note that the changing of the error type applies to the `or_else` and `transform_or` overloads only.

Other Languages

Rust provides a result type with similar functions monadic functions.

<https://doc.rust-lang.org/std/result/enum.Result.html>

Implementations

This has been implemented by Sy Brand

<https://github.com/TartanLlama/expected> with [documentation here](#). The author is unaware of another expected implementation that adds these functions.

Wording

Feature Test Macro

Update the `__cpp_lib_expected` feature test macro from P0323. Although this could be considered optional since expected is not yet a c++ feature in any implementations.

Class template `expected` [`expected.expected`]

After `value_or` functions add the new functions:

```
template<class U> constexpr T value_or(U&&) &&;

// [expected.monadic], monadic operations
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const&;
template <class F> constexpr auto and_then(F&& f) const&&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const&;
template <class F> constexpr auto or_else(F&& f) const&&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const&;
template <class F> constexpr auto transform(F&& f) const&&;
template <class F> constexpr auto transform_or(F&& f) &;
template <class F> constexpr auto transform_or(F&& f) &&;
template <class F> constexpr auto transform_or(F&& f) const&;
template <class F> constexpr auto transform_or(F&& f) const&&;
// [expected.mod], modifiers
```

Add a new sub-clause Monadic operations for [expected.monadic] between [expected.observe] and [expected.equality]

1 `template <class F> constexpr auto and_then(F&& f) &;`

2 `template <class F> constexpr auto and_then(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(value())>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E and U model `copy_constructible`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {
    return invoke(std::forward<F>(f), value());
}
else {
    return *this;
}
```

1 `template <class F> constexpr auto and_then(F&& f) &&;`

2 `template <class F> constexpr auto and_then(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(value())>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E and U model `move_constructible`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {
    return invoke(std::forward<F>(f), std::move(value()));
}
else {
    return std::move(*this);
}
```

1 `template <class F> constexpr auto or_else(F&& f) &;`

2 `template <class F> constexpr auto or_else(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::value_type, T>` is true. U and E model `copy_constructible`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {
    return U(value());
} else {
    return invoke(std::forward<F>(), error());
}
```

1 `template <class F> constexpr auto or_else(F&& f) &&;`

2 `template <class F> constexpr auto or_else(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::value_type, T>` is true. U and E model `move_constructible`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {
    return std::move(*this);
}
else {
    return invoke(std::forward<F>(f)(std::move(error())));
}
```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E and U model `copy_constructible`.

Effects: Equivalent to:

```
if (*this) {
    return expected<U,E>(in_place, invoke(std::forward<F>(f), value()));
}
else {
    return *this;
}
```

1 `template <class F> constexpr auto transform(F&& f) &&;`

2 `template <class F> constexpr auto transform(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E and U model `move_constructible`.

Effects: Equivalent to:

```
if (*this) {
    return expected<U,E>(in_place, invoke(std::forward<F>(f), std::move(
})
else {
    return std::move(*this);
}
```

1 `template <class F> constexpr auto transform_or(F&& f) &;`

2 `template <class F> constexpr auto transform_or(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::value_type, T>` is true. U and E model `copy_constructible`.

Effects: Equivalent to:

```
if (*this) {
    return *this;
}
else {
    return invoke(std::forward<F>(f)(error()));
}
```

1 `template <class F> constexpr auto transform_or(F&& f) &&;`

2 `template <class F> constexpr auto transform_or(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::value_type, T>` is true. U and E model `move_constructible`.

Effects: Equivalent to:

```
if (*this) {
    return std::move(*this);
}
else {
    return invoke(std::forward<F>(f)(std::move(error())));
}
```

Partial specialization of expected for void types [expected.void]

After 0.0.8.6, observers – add

```
// 0.0.8.7, monadic
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const&;
template <class F> constexpr auto and_then(F&& f) const&&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const&;
template <class F> constexpr auto or_else(F&& f) const&&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const&;
template <class F> constexpr auto transform(F&& f) const&&;
template <class F> constexpr auto transform_or(F&& f) &;
template <class F> constexpr auto transform_or(F&& f) &&;
template <class F> constexpr auto transform_or(F&& f) const&;
template <class F> constexpr auto transform_or(F&& f) const&&;
```

Partial specialization of expected for void

[expected.void.observe]

After section 8.5 Observers [expected.void.observe]

Add section 8.6 Monadic [expected.void.monadic]

```
1 template <class F> constexpr auto and_then(F&& f) &;
```

```
2 template <class F> constexpr auto and_then(F&& f) const&;
```

```
3 template <class F> constexpr auto and_then(F&& f) &&;
```

```
4 template <class F> constexpr auto and_then(F&& f) const&&;
```

Let U be `remove_cvref_t<invoke_result_t<F,void>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads. [Note: U will implicitly model either copy or move constructible since U is `expected<void, E>` – end note].

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {  
    return std::move(invoke(std::forward<F>(f)()));  
}  
else {  
    return std::move(*this);  
}
```

```
1 template <class F> constexpr expected or_else(F&& f) &;
```

```
2 template <class F> constexpr expected or_else(F&& f) const&;
```

3 `template <class F> constexpr expected or_else(F&& f) &&;`

4 `template <class F> constexpr expected or_else(F&& f) const &&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (*this) {
    return std::move(*this);
}
else {
    return invoke(std::forward<F>(f)());
}
```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const&;`

3 `template <class F> constexpr auto transform(F&& f) &&;`

4 `template <class F> constexpr auto transform(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F,void>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Effects: Equivalent to:

```

if (*this) {
    return expected<U,E>(in_place, invoke(std::forward<F>(f)()));
}
else {
    return std::move(*this);
}

```

1 `template <class F> constexpr auto transform_or(F&& f) &;`

2 `template <class F> constexpr auto transform_or(F&& f) const&;`

3 `template <class F> constexpr auto transform_or(F&& f) &&;`

4 `template <class F> constexpr auto transform_or(F&& f) &&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```

if (*this) {
    return std::move(*this);
}
else {
    return std::move(invoke(std::forward<F>(f)(error())));
}

```

Acknowledgements

- Thanks to Sy Brand for the original work on optional and expected.
- Thanks to Broneck Kozicki for early feedback.
- Thanks to Arthur O'Dwyer for design and wording feedback.

- Thanks to Barry Revzin for design and wording feedback.

Revision History

Version	Date	Changes
0	2021-12-12	Initial Draft
1	2022-02-10	added <code>transform_or</code>
		allow <code>transform</code> on <code>expected<void, E></code>
		updated status of P0323 to plenary approved
		improved examples
		added missing overloads for <code>or_else</code>
		<code>or_else</code> signature and wording fixes
		added design discussion of changing return types
		variety of wording fixes

References

1. [P0798] Sy Brand "Monadic Functions for `std::optional`"
<https://wg21.link/P0798>
2. C++ draft [optional.nomadic] <http://eel.is/c++draft/optional.monadic>
3. [P0323] Vicente Botet, JF Bastien, Jonathan Wakely `std::expected`
<https://wg21.link/P0323>
4. Sy Brand `expected` <https://github.com/TartanLlama/expected>
5. Sy Brand `expected` docs
<https://tl.tartanllama.xyz/en/latest/api/expected.html#tl-expected>
6. Rust Result <https://doc.rust-lang.org/std/result/enum.Result.html>
7. More complete examples <https://godbolt.org/z/va5fzx11f>